Elaborado por: Henry Antonio Mendoza Puerta

Ejercicio 01 — ViajaFácil

Índice

Haga clic (en Word: Ctrl + clic) sobre un paso para ir a él. Al final de cada paso encontrará el enlace "↑ Volver al índice".

Paso 1. Analizar el enunciado

Paso 2. Crear el proyecto y los paquetes en NetBeans

Paso 3. Copiar la clase externa SistemaCuposExterno (paquete com.usil.adapter)

Paso 4. Crear el modelo Reserva (paquete com.usil.modelo)

Paso 5. Crear la interfaz ProveedorCupos (paquete com.usil.adapter)

Paso 6. Crear el adaptador AdaptadorCupos (paquete com.usil.adapter)

Paso 7. Crear los subsistemas (paquete com.usil.subsistema)

Paso 8. Crear la fachada SistemaReservas (paquete com.usil.facade)

Paso 9. Crear el Main (paquete com.usil)

Paso 10. Ejecutar y comparar con el resultado esperado

¿Qué vamos a construir?

Un programa en Java que **registra reservas de paquetes turísticos**, **calcula el total** de cada una, **consulta los cupos** disponibles de cada destino en un sistema externo y muestra un **reporte en consola** usando lambdas y streams. Aplicaremos dos patrones:

- **Adapter:** el sistema externo devuelve un texto como CUSCO:12:ALTA, pero nuestro sistema necesita un número entero de cupos. El adaptador hace esa traducción.
- **Facade:** el Main no debe conocer al validador, a la calculadora ni al registro. Una sola clase fachada (SistemaReservas) ofrece todas las operaciones.

Clases que debe crear

Paquete	Clase	Rol
com.usil	Main	Registra las reservas de prueba y muestra el reporte
com.usil.modelo	Reserva	Única clase de modelo (solo datos)
com.usil.adapter	SistemaCuposExterno	Sistema externo (clase dada, no se modifica) — Adaptee
com.usil.adapter	ProveedorCupos	Interfaz que espera nuestro sistema — Target
com.usil.adapter	AdaptadorCupos	Convierte el texto externo en un entero — Adapter
com.usil.subsistema	ValidadorReserva	Valida los datos de la reserva
com.usil.subsistema	CalculadoraPrecio	Calcula el total (con descuento de grupo)
com.usil.subsistema	RegistroReservas	Guarda las reservas registradas
com.usil.facade	SistemaReservas	Fachada: único punto de acceso — Facade

Desarrollo paso a paso

Paso 1. Analizar el enunciado

Antes de programar, responda estas preguntas con el enunciado en la mano:

Pregunta	Respuesta
¿Qué datos tiene una reserva?	Código*, destino*, cliente*, número de personas* (int) y precio por persona (double).
¿Cuándo se rechaza una reserva?	Si el código, el destino o el cliente están vacíos, o si las personas no son mayores que 0.
¿Cómo se calcula el total?	personas × precio. Si son 4 personas o más (grupo), se descuenta el 10 %.
¿De dónde salen los cupos?	De SistemaCuposExterno, que devuelve un texto destino:cupos:temporada.
¿Qué patrón resuelve el formato incompatible?	Adapter.
¿Qué patrón simplifica el uso del sistema?	Facade.
¿Qué debe mostrar el programa?	Mensaje de rechazo, lista ordenada por código (cliente, destino, total, cupos), cantidad, suma de totales y reservas de grupo.

↑ Volver al índice

Paso 2. Crear el proyecto y los paquetes en NetBeans

1. Menú **File → New Project → Java with Maven → Java Application** y pulse Next.
2. Project Name: **Ejercicio01**. Group Id: **com.usil**. Finish.
3. NetBeans crea una clase de ejemplo (App). **Elimínela**; usaremos nuestro propio Main.
4. Clic derecho sobre **Source Packages → New → Java Package** y cree, uno por uno: com.usil.modelo, com.usil.adapter, com.usil.subsistema y com.usil.facade.
5. Clic derecho sobre el paquete **com.usil → New → Java Class** y cree la clase **Main** (la crearemos al final, en el Paso 9).

↑ Volver al índice

Paso 3. Copiar la clase externa SistemaCuposExterno (paquete com.usil.adapter)

El enunciado entrega esta clase ya escrita: **cópiela tal cual y no la modifique**. Representa un sistema de otra empresa que nosotros no controlamos. Observe que devuelve un texto con tres datos separados por dos puntos.

SistemaCuposExterno.java

```
package com.usil.adapter;

import java.util.Map;

public class SistemaCuposExterno {

    private static final Map<String, String> CUPOS = Map.of(
        "CUSCO", "CUSCO:12:ALTA",
        "AREQUIPA", "AREQUIPA:8:MEDIA",
        "PARACAS", "PARACAS:20:BAJA");

    public String consultar(String destino) {
        return CUPOS.get(destino);
```

```
    }
}
```

Puntos clave del código

- Map.of(...) guarda un texto por destino, por ejemplo CUSCO:12:ALTA (destino:cupos:temporada).
- consultar(destino) devuelve ese texto, o null si el destino no existe.
- Esta clase es el **Adaptee**: la que tiene la interfaz incompatible.

↑ Volver al índice

Paso 4. Crear el modelo Reserva (paquete com.usil.modelo)

Es la **única clase de modelo**: solo guarda datos. No tiene reglas ni cálculos. Atributos privados, un constructor y los getters.

Reserva.java

```
package com.usil.modelo;

public class Reserva {

    private final String codigo;
    private final String destino;
    private final String cliente;
    private final int numeroPersonas;
    private final double precioPorPersona;

    public Reserva(String codigo, String destino, String cliente, int numeroPersonas,
double precioPorPersona) {
        this.codigo = codigo;
        this.destino = destino;
        this.cliente = cliente;
        this.numeroPersonas = numeroPersonas;
        this.precioPorPersona = precioPorPersona;
    }

    public String getCodigo() { return codigo; }
    public String getDestino() { return destino; }
    public String getCliente() { return cliente; }
    public int getNumeroPersonas() { return numeroPersonas; }
    public double getPrecioPorPersona() { return precioPorPersona; }
}
```

Puntos clave del código

- Los atributos son private final: una vez creada, la reserva no cambia.
- NetBeans puede generar el constructor y los getters: clic derecho en el código → **Insert Code… → Constructor / Getter**.
- Las reglas (validar, calcular el total) **no** van aquí: irán en los subsistemas.

↑ Volver al índice

Paso 5. Crear la interfaz ProveedorCupos (paquete com.usil.adapter)

Define lo que **nuestro sistema** necesita: dado un destino, obtener los cupos como número entero. Es el **Target** del patrón Adapter. Para crearla: New → Java Interface.

ProveedorCupos.java

```
package com.usil.adapter;

public interface ProveedorCupos {

    int consultarCupos(String destino);
}
```

Puntos clave del código

- Nuestro sistema solo conocerá esta interfaz, nunca el texto del sistema externo.
- Gracias a ella, mañana se podría cambiar el sistema externo sin tocar el resto del programa.

↑ Volver al índice

Paso 6. Crear el adaptador AdaptadorCupos (paquete com.usil.adapter)

Es el **Adapter**: implementa ProveedorCupos y usa por dentro a SistemaCuposExterno. Su trabajo es traducir el texto CUSCO:12:ALTA al número 12.

AdaptadorCupos.java

```
package com.usil.adapter;

public class AdaptadorCupos implements ProveedorCupos {

    private final SistemaCuposExterno sistemaExterno;

    public AdaptadorCupos(SistemaCuposExterno sistemaExterno) {
        this.sistemaExterno = sistemaExterno;
    }

    @Override
    public int consultarCupos(String destino) {
        String texto = sistemaExterno.consultar(destino);
        if (texto == null) {
            return 0;
        }
        String[] partes = texto.split(":");
        return Integer.parseInt(partes[1]);
    }
}
```

Puntos clave del código

- implements ProveedorCupos: cumple lo que espera nuestro sistema.
- Recibe el SistemaCuposExterno por el constructor y lo guarda en un atributo (asociación).
- texto.split(":") separa el texto en partes: ["CUSCO", "12", "ALTA"]. La posición 1 contiene los cupos.
- Integer.parseInt(partes[1]) convierte "12" (texto) en 12 (entero).
- Si el destino no existe, el sistema externo devuelve null; el adaptador responde 0 cupos para evitar un error.

↑ Volver al índice

Paso 7. Crear los subsistemas (paquete com.usil.subsistema)

Son tres clases pequeñas, cada una con **una sola responsabilidad**. La fachada las usará, pero el Main no.

7.1 ValidadorReserva

ValidadorReserva.java

```
package com.usil.subsistema;

import com.usil.modelo.Reserva;

public class ValidadorReserva {

    public boolean esValida(Reserva reserva) {
        return noVacio(reserva.getCodigo())
                && noVacio(reserva.getDestino())
                && noVacio(reserva.getCliente())
                && reserva.getNumeroPersonas() > 0;
    }

    private boolean noVacio(String texto) {
        return texto != null && !texto.isBlank();
    }
}
```

- esValida(...) devuelve true solo si código, destino y cliente no están vacíos **y** las personas son mayores que 0.
- isBlank() detecta texto vacío o solo con espacios.
- noVacio es un método privado auxiliar para no repetir código.

7.2 CalculadoraPrecio

CalculadoraPrecio.java

```
package com.usil.subsistema;

import com.usil.modelo.Reserva;

public class CalculadoraPrecio {

    private static final int PERSONAS_GRUPO = 4;
    private static final double DESCUENTO_GRUPO = 0.10;

    public double calcularTotal(Reserva reserva) {
        double total = reserva.getNumeroPersonas() * reserva.getPrecioPorPersona();
        if (reserva.getNumeroPersonas() >= PERSONAS_GRUPO) {
            total = total - total * DESCUENTO_GRUPO;
        }
        return total;
    }
}
```

- Total base = personas × precio por persona.
- Si las personas son 4 o más (PERSONAS_GRUPO), se resta el 10 % (DESCUENTO_GRUPO).
- Las constantes (static final) evitan “números mágicos” en el código.

7.3 RegistroReservas

RegistroReservas.java

```
package com.usil.subsistema;

import com.usil.modelo.Reserva;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class RegistroReservas {

    private final List<Reserva> reservas = new ArrayList<>();

    public void agregar(Reserva reserva) {
        reservas.add(reserva);
    }

    public List<Reserva> obtenerTodas() {
        return Collections.unmodifiableList(reservas);
    }
}
```

- Guarda las reservas en una lista (ArrayList).
- obtenerTodas() devuelve una lista **no modificable**: nadie puede agregar reservas sin pasar por agregar(...).

↑ Volver al índice

Paso 8. Crear la fachada SistemaReservas (paquete com.usil.facade)

Es el **Facade**: el único punto de acceso del usuario. Crea los subsistemas y el adaptador, y ofrece cuatro operaciones sencillas: registrar reserva, validar reserva, calcular total y consultar cupos (más obtenerReservas para el reporte).

SistemaReservas.java

```
package com.usil.facade;

import com.usil.adapter.AdaptadorCupos;
import com.usil.adapter.ProveedorCupos;
import com.usil.adapter.SistemaCuposExterno;
import com.usil.modelo.Reserva;
import com.usil.subsistema.CalculadoraPrecio;
import com.usil.subsistema.RegistroReservas;
import com.usil.subsistema.ValidadorReserva;
import java.util.List;

public class SistemaReservas {

    private final ValidadorReserva validador = new ValidadorReserva();
    private final CalculadoraPrecio calculadora = new CalculadoraPrecio();
    private final RegistroReservas registro = new RegistroReservas();
    private final ProveedorCupos proveedorCupos = new AdaptadorCupos(new
SistemaCuposExterno());

    public boolean registrarReserva(Reserva reserva) {
        if (!validarReserva(reserva)) {
            return false;
        }
        registro.agregar(reserva);
        return true;
    }

    public boolean validarReserva(Reserva reserva) {
        return validador.esValida(reserva);
    }
```

```
    public double calcularTotal(Reserva reserva) {
        return calculadora.calcularTotal(reserva);
    }

    public int consultarCupos(String destino) {
        return proveedorCupos.consultarCupos(destino);
    }

    public List<Reserva> obtenerReservas() {
        return registro.obtenerTodas();
    }
}
```

Puntos clave del código

- Los atributos validador, calculadora y registro son los subsistemas; proveedorCupos es la interfaz (el adaptador).
- registrarReserva primero valida; si no es válida devuelve false y **no** la guarda; si es válida la guarda y devuelve true.
- Cada método **delega** el trabajo en el subsistema correspondiente: la fachada casi no tiene lógica propia.
- El adaptador se asigna a una variable de tipo ProveedorCupos (la interfaz), no de tipo AdaptadorCupos.

↑ Volver al índice

Paso 9. Crear el Main (paquete com.usil)

El Main **solo usa la fachada** (y el modelo Reserva para crear los datos de prueba). Observe que no hay ningún for ni while: todo el reporte se hace con lambdas y streams.

Main.java

```
package com.usil;

import com.usil.facade.SistemaReservas;
import com.usil.modelo.Reserva;
import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class Main {

    private static final int PERSONAS_GRUPO = 4;

    public static void main(String[] args) {
        SistemaReservas sistema = new SistemaReservas();

        List<Reserva> reservasDePrueba = List.of(
            new Reserva("RES-004", "CUSCO", "Ana Soto", 2, 450.00),
            new Reserva("RES-001", "AREQUIPA", "Luis Vega", 5, 300.00),
            new Reserva("RES-003", "PARACAS", "Rosa León", 0, 200.00),
            new Reserva("RES-002", "CUSCO", "Marta Gil", 4, 450.00),
            new Reserva("RES-005", "PARACAS", "Juan Pérez", 3, 180.00));

        reservasDePrueba.forEach(reserva -> {
            if (!sistema.registrarReserva(reserva)) {
                System.out.println("Reserva " + reserva.getCodigo() + " RECHAZADA: datos
no válidos.");
            }
        });

        System.out.println();
        System.out.println("=== RESERVAS REGISTRADAS (ordenadas por código) ===");
```

```
        sistema.obtenerReservas().stream()
            .sorted(Comparator.comparing(Reserva::getCodigo))
            .forEach(r -> System.out.printf(java.util.Locale.US,
                "%s | %-10s | %-9s | Total: S/ %.2f | Cupos: %d%n",
                r.getCodigo(), r.getCliente(), r.getDestino(),
                sistema.calcularTotal(r), sistema.consultarCupos(r.getDestino())));

        long cantidad = sistema.obtenerReservas().stream().count();
        double sumaTotales = sistema.obtenerReservas().stream()
            .mapToDouble(sistema::calcularTotal)
            .sum();
        String reservasGrupo = sistema.obtenerReservas().stream()
            .filter(r -> r.getNumeroPersonas() >= PERSONAS_GRUPO)
            .map(Reserva::getCodigo)
            .sorted()
            .collect(Collectors.joining(", "));

        System.out.println();
        System.out.println("Cantidad de reservas registradas: " + cantidad);
        System.out.printf(java.util.Locale.US, "Suma de totales: S/ %.2f%n", sumaTotales);
        System.out.println("Reservas de grupo (4 personas o más): " + reservasGrupo);
    }
}
```

Cómo funciona cada parte del reporte

Instrucción	Qué hace
List.of(new Reserva(...), ...)	Crea la lista con las 5 reservas de prueba, en el orden del enunciado.
reservasDePrueba.forEach(reserva -> { ... })	Recorre la lista con una lambda; si registrarReserva devuelve false, imprime el mensaje de rechazo.
stream().sorted(Comparator.comparing(Reserva::getCodigo))	Ordena las reservas por código sin modificar la lista original.
.forEach(r -> System.out.printf(...))	Imprime cada reserva con cliente, destino, total (fachada) y cupos (fachada → adaptador).
stream().count()	Cuenta las reservas registradas.
stream().mapToDouble(sistema::calcularTotal).sum()	Convierte cada reserva en su total (double) y los suma.
stream().filter(r -> r.getNumeroPersonas() >= 4)	Se queda solo con las reservas de grupo.
.map(Reserva::getCodigo).sorted().collect(Collectors.joining(", "))	Obtiene los códigos, los ordena y los une en un solo texto separado por comas.
Locale.US en printf	Fuerza el punto decimal (1350.00) sin importar la configuración regional.

↑ Volver al índice

Paso 10. Ejecutar y comparar con el resultado esperado

Clic derecho sobre **Main.java → Run File**. La consola de NetBeans debe mostrar exactamente esto:

Salida esperada en la consola

```
Reserva RES-003 RECHAZADA: datos no válidos.

=== RESERVAS REGISTRADAS (ordenadas por código) ===
RES-001 | Luis Vega  | AREQUIPA  | Total: S/ 1350.00 | Cupos: 8
RES-002 | Marta Gil  | CUSCO     | Total: S/ 1620.00 | Cupos: 12
RES-004 | Ana Soto   | CUSCO     | Total: S/ 900.00 | Cupos: 12
RES-005 | Juan Pérez | PARACAS   | Total: S/ 540.00 | Cupos: 20

Cantidad de reservas registradas: 4
Suma de totales: S/ 4410.00
Reservas de grupo (4 personas o más): RES-001, RES-002
```

Verifique estos puntos:

- RES-003 (0 personas) fue rechazada y no aparece en la lista.
- La lista está ordenada por código (RES-001, 002, 004, 005) aunque se registraron en otro orden.
- RES-001 (5 personas) y RES-002 (4 personas) tienen 10 % de descuento: 5×300=1500→1350 y 4×450=1800→1620.
- Los cupos coinciden con el sistema externo: AREQUIPA 8, CUSCO 12, PARACAS 20.
- Cantidad = 4 y suma = 4410.00 (1350 + 1620 + 900 + 540).

↑ Volver al índice

Diagrama de Clase

Diagrama de clases de la solución

